

Definición de tu Caso de Uso AI/LLM

Completa los 6 pasos durante la clase. Al cerrar tendrás el problema definido, el salto que quieres lograr y una hipótesis de solución. Escribe directamente en los recuadros amarillos. *Curso Proyecto Final AI Data Engineer · BSG Institute*

PASO 1 · Identifica al equipo y el contexto

Datos básicos del equipo y resumen en una sola frase del caso de uso elegido.

Equipo (integrantes) Nombres completos y rol que asumirá cada uno en el proyecto.	Miguel Mercado — <i>Rol: Data & AI Architect / Backend Developer</i> (responsable del diseño de la base de datos documental, la integración de microservicios, el motor vectorial pgvector y la lógica RAG del core).
Organización / sector Empresa real o caso del que toman los datos. Sector económico.	TI — Sector de Tecnología, Desarrollo de Software y Soporte de Aplicaciones TI.
Caso de uso en 1 frase Resumen en una sola frase: qué hace el sistema, para quién, sobre qué datos.	Asistente RAG multi-tenant inteligente (DocViz) que permite a los ingenieros de soporte técnico consultar código fuente, documentación técnica e historial de tareas en células organizativas de repositorios Git de forma ágil y segura

PASO 2 · Delimita el problema con el marco TADR

Llena las cinco celdas con precisión. Si una celda no puede llenarse con datos concretos, el caso aún no está listo.

Trigger Evento concreto que dispara la necesidad. Incluye frecuencia (cuántas veces al día/semana ocurre).	Asignación de un nuevo ticket de soporte de Nivel 2/3 (análisis de un error en código, duda sobre infraestructura, configuración de microservicios) a un ingeniero de soporte (frecuencia promedio: 3 a 5 veces al día por ingeniero).
Actor Persona específica que enfrenta el problema hoy. Rol granular, no un departamento abstracto.	Ingeniero de soporte de desarrollo / Ingeniero de soporte técnico de Nivel 2 y 3.
Tarea hoy Pasos concretos que el actor hace hoy para resolver la necesidad, sin idealizar.	El ingeniero abre el ticket, identifica los microservicios involucrados, clona manualmente los repositorios de Git correspondientes en su máquina local, busca palabras clave usando herramientas de IDE, busca manuales de arquitectura en repositorios S3 o wikis dispersas, y finalmente redacta la respuesta técnica.
Dolor (con números) Qué incomoda al actor en esa tarea, con magnitudes verificables (tiempo, errores, volumen).	45 minutos en promedio por ticket perdidos en búsqueda e indexación manual de información. Tasa de reincidencia o error de solución técnica del 12% debido a la falta de consulta de históricos de tareas similares resueltas previamente por

	otros miembros de la célula.
Resultado deseado (con números) Qué situación querría el actor en lugar de la actual, también con magnitudes medibles.	Reducir el tiempo promedio de resolución a menos de 15 minutos (reducción del 66%) y reducir la tasa de error técnico en las respuestas por debajo del 4% .

PASO 3 · Descompón el proceso y marca dónde interviene la IA

El paso 2 resume el problema en un solo momento. Aquí despliegas la secuencia completa: 4 a 6 etapas concretas del proceso actual, quién actúa en cada una, y si tu sistema interviene o no. Las etapas sin intervención siguen siendo manuales — con esto ya defines el alcance real de tu solución.

Etapas	Cómo es hoy (AS IS)	¿Interviene la IA? Qué cambiaría (TO BE)
Etapas 1 Quién actúa y qué hace en esta etapa.	<i>El ingeniero crea una nueva tarea de soporte y busca los repositorios afectados a mano.</i>	<i>No interviene. El flujo de creación de tareas se gestiona mediante la interfaz gráfica del panel de DocViz.</i>
Etapas 2 Quién actúa y qué hace en esta etapa.	<i>El ingeniero clona e indexa mentalmente el código y documentos en su máquina local.</i>	<i>Sí interviene. Al añadir el repositorio Git a la célula en DocViz, el sistema clona de forma automática con JGit, segmenta los archivos en chunks y calcula embeddings vectoriales usando OpenAI/Gemini, guardándolos en pgvector.</i>
Etapas 3 Quién actúa y qué hace en esta etapa.	<i>El ingeniero busca de forma manual líneas de código y documentación técnica para responder al ticket.</i>	<i>Sí interviene. El sistema ofrece una interfaz de chat interactiva integrada con RAG que responde consultas de código basándose en el repositorio e historial indexados.</i>
Etapas 4 Quién actúa y qué hace en esta etapa.	<i>El ingeniero resume manualmente el chat en su ticket final para mantener el historial.</i>	<i>Sí interviene. Un servicio resumidor automático procesa la conversación de chat (cada 20 mensajes) para generar un rolling summary interactivo persistido en MongoDB.</i>
Etapas 5 Quién actúa y qué hace en esta etapa.	<i>El ingeniero elimina las carpetas locales clonadas para liberar espacio de forma manual.</i>	<i>Sí interviene. Al borrar una célula organizativa, el sistema ejecuta un borrado en cascada para detectar repositorios huérfanos y elimina automáticamente su almacenamiento local/S3, metadatos y embeddings vectoriales de pgvector.</i>

PASO 4 · Define el tipo de tarea AI

Marca un solo tipo. Si dudas entre dos, vuelve al paso 2. Justifica brevemente por qué descartas los demás.

☐ Generación libre Producir contenido nuevo a partir de instrucciones. <i>Ej. Redactar un correo o crear una propuesta a partir de variables.</i>	☒ IXI RAG (recuperación aumentada) Responder preguntas a partir de un corpus específico de documentos. <i>Ej. Asistente que responde dudas usando los manuales internos.</i>
☐ Extracción estructurada Convertir texto en un objeto con campos tipados. <i>Ej. Leer un correo y completar la ficha del CRM (cliente, fecha, ítems).</i>	☐ Clasificación semántica Asignar una etiqueta de un conjunto cerrado a un texto. <i>Ej. Clasificar tickets de soporte por departamento responsable.</i>

Summarization Condensar contenido extenso en una versión más corta con criterios específicos. <i>Ej. Resumir actas de reuniones para briefing ejecutivo.</i>	Agente (ejecución agéntica) Tomar decisiones y ejecutar acciones sobre sistemas externos. <i>Ej. Crear ticket, consultar API y luego decidir el siguiente paso.</i>
Justificación Por qué el tipo elegido y no los otros cinco. Explica brevemente por qué descartas las alternativas.	- Se selecciona RAG porque la funcionalidad central es responder preguntas técnicas Q&A en lenguaje natural usando un corpus de datos específico (repositorios Git del cliente, archivos S3 y base de datos relacional). - Se descartan los demás porque: <i>Generación libre</i>: No deseamos respuestas "creativas" que puedan alucinar; deben ceñirse estrictamente al código del cliente. <i>Extracción estructurada</i>: La interacción del programador/soporte es conversacional, no la extracción de campos tipados fijos en un JSON. <i>Clasificación</i>: No buscamos categorizar el texto, sino resolver dudas de programación. <i>Summarization</i>: El rolling summary es una función secundaria, no la razón principal del proyecto. <i>Agente</i>: El sistema no toma decisiones multinivel ni ejecuta acciones autónomas en bases de datos externas de producción.

PASO 5 · Escribe el contrato de entrada y salida

Describe qué recibe y qué produce el sistema. Sé específico con tipos de campo y formato.

Entrada (qué recibe) Formato y fuente de los datos que el sistema procesa. Incluye tipo de archivo y origen. <i>Ej. Mensaje de correo (texto/HTML) más 0-5 adjuntos PDF, recibido vía API de Microsoft Graph. Incluye remitente, asunto, fecha y ID único.</i>	Mensaje en lenguaje natural enviado por el usuario (string), taskId de la tarea de soporte (string/UUID), conversacionId (string/UUID), y opcionalmente menciones estructuradas a archivos específicos como @ [repo:filename].
Salida (esquema con campos y tipos) Esquema completo de la salida con nombres y tipos de cada campo. Incluye scores de confianza y comportamiento ante incertidumbre. <i>Ej. JSON con cliente_id (string null), cliente_nombre (string), fecha_requerida (date), productos (array), score_global (float 0-1), campos_baja_confianza (array).</i>	Stream progresivo del contenido de la respuesta en formato SSE (text/event-stream). Al completarse el stream, se almacena en la colección de MongoDB un objeto JSON de tipo MensajeChatDocumento con el siguiente esquema: - texto_entrada (string): Pregunta formulada por el usuario. - texto_respuesta (string): Respuesta del LLM contextualizada con RAG. - fecha_respuesta (date): Fecha del servidor. - hora_respuesta (date): Timestamp detallado de la respuesta.
Ejemplo concreto entrada → salida	Entrada: "@[repo:docker-compose.yml] describe los servicios principales"

Un par entrada-salida realista que ilustre el comportamiento del sistema.

Ej. Entrada: correo de XYZ Industrial pidiendo 200 cajas de tornillos M8 para el 15/08, con PDF adjunto. Salida: cliente_id 'XYZ-2031', productos con 2 entradas, score 0.94, citas a líneas de origen.

- **Salida:**

Stream SSE

deltas: data:El data:archivo data:docker data:contiene ...

Persistencia final:

```
{
  "texto_entrada": "@[repo:docker-compose.yml] describe los servicios principales",
  "texto_respuesta": "El archivo docker-compose.yml define 6 servicios: bsg-mongo (base de datos de tareas), bsg-postgres-docviz (base relacional y vectorial con pgvector), bsg-redis (memoria intermedia), bsg-localstack (emulación local de AWS S3 y DynamoDB), bsg-eureka (servidor de registro y descubrimiento) y bsg-soporte-rag-mt (núcleo del motor RAG multi-tenant).",
  "fecha_respuesta": "2026-08-16T00:00:00.000Z",
  "hora_respuesta": "2026-08-16T12:00:01.325Z"
}
```

PASO 6 · Redacta tu approach de solución

Un párrafo de 4 a 6 frases, en prosa — no una lista. Cierra la guía conectando lo ya definido: qué construye el sistema, qué tipo de tarea AI usa, y en qué etapas del proceso del paso 3 interviene.

Guíate con: ¿Qué hace el sistema y con qué datos? ¿Qué tipo de tarea AI del paso 4 usa? ¿En qué etapas del paso 3 interviene y en cuáles no? ¿Qué queda explícitamente fuera de alcance por ahora?

El sistema DocViz es un asistente interactivo multi-tenant de soporte de software que automatiza la búsqueda de conocimiento en repositorios de código y documentación técnica. Su backend principal está desarrollado en Java con Spring Boot (soporte-rag-mt) que integra Spring AI para la comunicación con LLMs de OpenAI/Gemini y utiliza un motor relacional PostgreSQL con la extensión pgvector para el almacenamiento y la búsqueda de embeddings de alta fidelidad. La solución intervendrá agilizando la indexación automática de repositorios, facilitando respuestas enriquecidas mediante una conversación RAG y unificando el seguimiento con un resumidor de chat persistente en MongoDB. Quedan fuera del alcance del MVP las acciones automatizadas en ambientes de producción (como deploys de infraestructura o committear parches de código correctivos de manera autónoma sin supervisión humana).

VALIDACIÓN FINAL · Antes de cerrar la sesión

Marca cada casilla. Si alguna queda en blanco, vuelve al paso correspondiente antes de avanzar a la sesión 2.

- [x] Equipo, organización y caso de uso en una frase están completos (paso 1).
- [x] El problema tiene Trigger, Actor, Tarea, Dolor y Resultado con magnitudes verificables (paso 2).
- [x] El proceso está descompuesto en 5 etapas concretas y cada una indica si la IA interviene o no (paso 3).
- [x] El tipo de tarea AI es uno solo, justificado contra los otros cinco (paso 4).
- [x] El contrato I/O tiene esquema concreto y al menos un ejemplo (paso 5).

[x] El approach de solución es un párrafo (no una lista) que conecta tipo de tarea, contrato I/O y las etapas del paso 3 donde interviene (paso 6).

[x] Una persona externa al equipo puede leer la guía completa y entender qué van a construir y por qué.

Fecha de cierre	Firma del equipo
08/09/2026	